ROOTIFY PROJECT

Complete Codebase Audit, Architecture & Developer Documentation

Application: Rootify (Authentic Goan Regional Marketplace)

Frameworks: React 19 • TypeScript • Vite • Tailwind CSS • React Router 7

Architecture: Single Page App with Browser-Persisted Storage **Date:** October 2026

Executive Summary

This document provides a comprehensive, file-by-file audit and technical reference for the **Rootify** codebase. Rootify is a regional food marketplace designed specifically for Goa, India, connecting customers directly with local Goan home-chefs, artisanal bakers, self-help groups (SHGs / Mahila Mandals), and heritage spice plantations. The platform incorporates single-vendor artisanal fulfillment, origin traceability, Taluka-level delivery routing, and dedicated portals for **Customers, Vendors, Delivery Partners, and Admins**.

1. Complete Project Overview

Rootify is structured as a modern, high-performance client-side application. Every component, state model, and service module has been designed following modular best practices.

Technology	Version & Role	Purpose in Rootify Codebase
React	`19.2.8` (Core Framework)	Component-driven UI, virtual DOM rendering, and standard hooks (`useState` , `useEffect` , `useCallback` , `useContext`).
TypeScript	`~6.0.2` (Language)	Strict static type system defining all contracts across users, products, categories, orders, reviews, locations, and chat messages in `src/types/index.ts` .
Vite	`8.3.0` (Bundler / Dev Server)	Ultra-fast Hot Module Replacement (HMR) local development server and Rollup-powered production asset bundler.
React Router DOM	`7.18.4` (Client Routing)	Declarative SPA routing supporting 24 routes across public marketplace, auth, vendor portal, delivery partner portal, and admin desk.
Tailwind CSS	`3.4.17` + PostCSS (Styling)	Custom-tokenized design system providing Goan forest greens (`brand`), warm terracotta (`terracotta`), sand tones (`sand`), and dark charcoal typography.
Lucide React	`1.51.0` (Iconography)	Vector SVG icon set used across navigation headers, marketplace filters, order status timelines, and dashboard KPI cards.
Canvas Confetti	`1.9.4` (Visual FX)	Celebratory particle animation triggered upon successful checkout and order placement.

Technology	Version & Role	Purpose in Rootify Codebase
Oxlint	`1.81.0` (Code Quality)	Rust-based linter configured via `.oxlintrc.json` for instant static analysis.

Backend & Database Status

Current State: The codebase is currently **frontend-only**. No remote backend server or SQL/NoSQL database is connected. Instead, all data operations are encapsulated in a robust **Service Layer** (`src/services/`) backed by **Browser** `localStorage` . On first launch, initial seed data is hydrated into storage. Adding products, editing prices, creating orders, writing reviews, and updating delivery statuses write immediately to `localStorage` and sync across React contexts in real-time.

2. Complete Folder and File Structure

Below is the complete, exact directory hierarchy of the Rootify codebase:

```
Dummy/
├── .gitignore                          # Git version control ignore rules
├── .oxlintrc.json                      # Oxlint linter configuration
├── README.md                           # Project boilerplate readme
├── index.html                          # HTML5 application root entry
├── package.json                        # NPM dependencies & run scripts
├── package-lock.json                   # Locked dependency tree
├── postcss.config.js                   # PostCSS plugins (Tailwind & Autoprefixer)
├── tailwind.config.js                  # Custom Rootify color palette & design tokens
├── tsconfig.json                       # TypeScript project references root
├── tsconfig.app.json                   # App source TypeScript compiler options
├── tsconfig.node.json                  # Vite configuration TypeScript rules
├── vite.config.ts                      # Vite build & plugin configuration
├── public/                             # Static public web assets
│   ├── favicon.svg                     # Browser favicon
│   └── icons.svg                       # Auxiliary SVG asset
└── src/                                # Application source code
    ├── App.css                         # Legacy template stylesheet (unused by Rootify)
    ├── App.tsx                         # Root component, router & context provider tree
    ├── index.css                       # Global styles, fonts, scrollbars & glassmorphism
    ├── main.tsx                        # React DOM client mounting entry point
    ├── assets/                         # Bundled static media assets (hero.png, react.svg, vite.svg)
    ├── types/                          # TypeScript definitions & data contracts
    │   └── index.ts                    # User, Product, Vendor, Order, Review, Chat models
    ├── data/                           # Initial mock datasets for Goan marketplace
    │   ├── categories.ts               # 9 authentic Goan food categories
    │   ├── locations.ts                # 10 Goa hubs & 11 Taluka definitions
    │   ├── messages.ts                 # Sample chat messages, issues & settlement records
    │   ├── orders.ts                   # Initial sample orders with timelines
    │   ├── products.ts                 # 16 authentic Goan products with origin details
    │   ├── reviews.ts                  # Verified purchase reviews & star ratings
    │   ├── users.ts                    # 4 demo accounts (Customer, Vendor, Delivery, Admin)
    │   └── vendors.ts                  # 7 authentic Goan producers & SHGs
    ├── services/                       # Data access layer & storage drivers
    │   ├── adminService.ts             # Admin operations (disputes, users, settlements)
    │   ├── authService.ts              # Authentication & instant demo role-switching
    │   ├── chatService.ts              # Delivery partner chat logic
    │   ├── orderService.ts             # Order lifecycle, status transitions & settlements
    │   ├── productService.ts           # Product CRUD, search & multi-facet filtering
    │   ├── reviewService.ts            # Verified reviews & dynamic rating recalculation
    │   ├── storage.ts                  # LocalStorage schema initialization & read/write
    │   └── vendorService.ts            # Vendor registration, verification & profiles
    ├── context/                        # React Context state providers
    │   ├── AuthContext.tsx             # Active user state, auth methods & demo role switch
    │   ├── CartContext.tsx             # Single-vendor cart state, quantity & fees
    │   ├── LocationContext.tsx         # Active Goa location & serviceability check
    │   ├── OrderContext.tsx            # Orders state, real-time sync & placement
    │   └── ToastContext.tsx            # Global toast notifications banner system
    ├── components/                     # Reusable UI component library
    │   ├── common/                     # Cross-cutting layout & utility components
    │   │   ├── EmptyState.tsx          # Reusable zero-data illustration & action prompt
    │   │   ├── Footer.tsx              # Marketplace footer with links, SHG note & copyright
    │   │   ├── Header.tsx              # Sticky navigation, search bar, location & cart badge
    │   │   ├── LocationModal.tsx       # Goa hub & Taluka selection dialog
    │   │   ├── MarketplaceLayout.tsx   # Wrapper combining Header, Footer, Nav & RoleSwitcher
    │   │   ├── MobileNav.tsx           # Mobile bottom sticky navigation bar
    │   │   ├── ProtectedRoute.tsx      # Role-based route guard & redirect handler
    │   │   └── RoleSwitcherBar.tsx     # Demo utility bar to toggle between 4 roles instantly
    │   ├── marketplace/                # Customer-facing shopping & product components
    │   │   ├── CategoryCard.tsx        # Visual category tile with dish counter
    │   │   ├── OrderTimeline.tsx       # Step-by-step order preparation & delivery tracker
    │   │   ├── ProductCard.tsx         # Product card with origin badge, rating & quantity
    │   │   ├── ReviewModal.tsx         # Modal dialog for rating and reviewing delivered items
    │   │   └── VendorCard.tsx          # Producer profile card with scores, village & badges
```

```
│   └── portals/                    # Dashboard shell for specialized portals
│       └── PortalLayout.tsx        # Common sidebar, mobile drawer & portal navigation
└── pages/                          # Routed view pages organized by role
    ├── admin/                      # 8 Admin views (Dashboard, Issues, Orders, Products, Reports, Set
    ├── auth/                       # 2 Auth views (LoginPage, SignupPage)
    ├── common/                     # 1 Common view (NotFoundPage)
    ├── customer/                   # 9 Customer views (Cart, Checkout, Profile, Explore, Home, Order[
    ├── delivery/                   # 3 Delivery Partner views (Chats, Dashboard, Profile)
    └── vendor/                     # 6 Vendor views (Dashboard, Earnings, Orders, Products, Profile,
```

3. Detailed File-by-File Inventory

3.1 Root Configuration & Infrastructure

File Path	Purpose & Contents	Dependencies & Usage
`package.json`	Declares runtime dependencies, devDependencies, and npm scripts (`dev`, `build`, `lint`, `preview`).	Used by npm and Vite bundler.
`vite.config.ts`	Configures Vite with React plugin (`@vitejs/plugin-react`).	Used during dev server startup and production bundling.
`index.html`	Root HTML5 template. Mounts `#root` div, imports Google Fonts (Outfit & Plus Jakarta Sans), and loads `src/main.tsx`.	Entry point fetched by the browser.
`tailwind.config.js`	Configures Tailwind CSS design tokens: `brand` green, `terracotta` coral, `sand` cream, custom shadows, and typography fonts.	Parsed by Tailwind compiler and used across all JSX files.
`src/index.css`	Global stylesheet. Declares Tailwind directives, base background (`#FAF8F3`), custom scrollbars, `.glass-header`, and utility classes.	Imported in `src/main.tsx`.
`src/App.css`	Legacy starter CSS from default Vite template. Unused by active Rootify components.	Not imported anywhere in the active app.
`src/main.tsx`	Bootstrap client entry file. Renders `<App />` inside React `StrictMode` into the DOM element `#root`.	Loaded by `index.html`.
`src/App.tsx`	Root component configuring the Context Provider hierarchy and registering all 24 client-side routes.	Wraps the entire application tree.

3.2 TypeScript Contracts & Seed Datasets

File Path	Purpose & Contents	Dependencies & Usage
`src/types/index.ts`	Core type definitions: `User`, `UserRole`, `Product`, `Vendor`, `Order`, `OrderStatus`, `GoaLocation`, `Review`, `ChatMessage`, `SettlementRecord`.	Imported by virtually every file in the codebase.
`src/data/categories.ts`	Exports 9 authentic Goan food categories (Goan Sweets, Traditional Foods, Spices, Bakery, Cashews, etc.).	Used by Home, Explore, and Vendor product creation pages.

File Path	Purpose & Contents	Dependencies & Usage
`src/data/locations.ts`	Exports 10 major Goa delivery hubs (Panaji, Mapusa, Margao, etc.) and all 11 Goan Talukas.	Used by LocationContext, checkout, and search filters.
`src/data/users.ts`	Exports 4 pre-configured demo user accounts (Customer, Vendor, Delivery, Admin).	Used by `storage.ts` and `authService.ts` for instant role switching.
`src/data/vendors.ts`	Exports 7 authentic Goan food producers, SHGs, and plantations with verification documents and stories.	Initial seed dataset for `vendorService.ts`.
`src/data/products.ts`	Exports 16 authentic Goan regional food dishes with detailed origin and traditional preparation metadata.	Initial seed dataset for `productService.ts`.
`src/data/orders.ts`	Exports sample orders with 7-step timeline statuses, customer items, and delivery partner assignments.	Initial seed dataset for `orderService.ts`.
`src/data/reviews.ts`	Exports initial verified customer reviews and ratings for Goan dishes.	Initial seed dataset for `reviewService.ts`.
`src/data/messages.ts`	Exports sample delivery chat conversations, dispute claims, and settlement payout records.	Used by `chatService.ts` and `adminService.ts`.

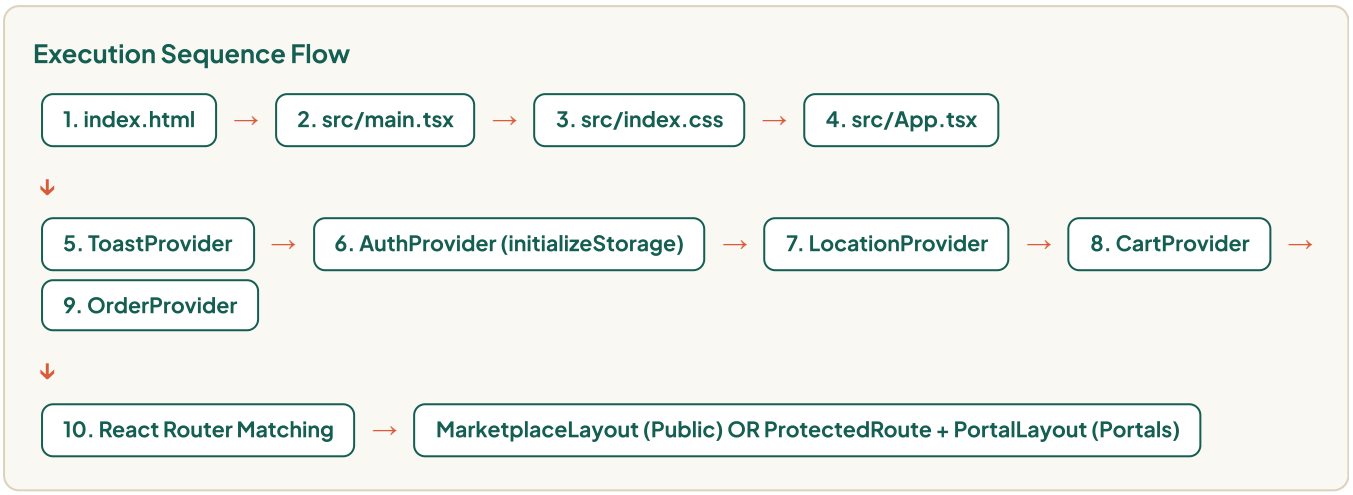
3.3 Services & React Context Layer

File Path	Purpose & Contents	Dependencies & Usage
`src/services/storage.ts`	Low-level browser `localStorage` manager. Handles schema keys and seeds mock data on first launch.	Imported by all mock services to read and write state.
`src/services/authService.ts`	Authentication logic, customer signup, profile updates, and demo role switching.	Consumed by `AuthContext.tsx`.
`src/services/productService.ts`	Product catalog queries, search, multi-facet filtering, and vendor CRUD operations.	Consumed by Home, Explore, ProductDetail, VendorProducts, AdminProducts.
`src/services/orderService.ts`	Order lifecycle manager. Enforces role-based state transitions and triggers settlement calculations.	Consumed by `OrderContext.tsx`.
`src/services/vendorService.ts`	Vendor onboarding, verification approvals/rejections, and producer profile updates.	Consumed by VendorRegister, VendorProfile, AdminVendors.
`src/services/reviewService.ts`	Verified reviews manager. Automatically recalculates product and vendor star ratings upon review submission.	Consumed by `OrderContext.tsx` and `ProductDetailPage.tsx`.
`src/services/chatService.ts`	Delivery partner direct chat service connecting drivers with kitchens and customers.	Consumed by `DeliveryChatsPage.tsx`.
`src/services/adminService.ts`	Platform admin operations: dispute resolutions, user directory, and vendor payout settlements.	Consumed by AdminIssues, AdminSettlements, AdminUsers.
`src/context/AuthContext.tsx`	Provides global authentication state, active role, and demo role switching functions.	Wraps application tree; consumed via `useAuth()`.
`src/context/CartContext.tsx`	Manages shopping cart with strict single-vendor artisanal conflict protection and fee calculation.	Consumed via `useCart()`.
`src/context/LocationContext.tsx`	Maintains active Goa delivery location and calculates vendor-to-customer Taluka serviceability.	Consumed via `useLocation()`.
`src/context/OrderContext.tsx`	Coordinates order placement, live status progression, and cross-tab storage synchronization.	Consumed via `useOrders()`.
`src/context/ToastContext.tsx`	Renders animated toast notifications (success, error, info) with auto-dismiss timers.	Consumed via `useToast()`.

3.4 Reusable UI Components & Layouts

File Path	Purpose & Contents	Dependencies & Usage
`components/common/Header.tsx`	Main sticky header with brand logo, Goa location selector, quick search input, and cart counter badge.	Used in `MarketplaceLayout`.
`components/common/Footer.tsx`	Marketplace footer with category links, Goan SHG mission note, and portal navigation shortcuts.	Used in `MarketplaceLayout`.
`components/common/MobileNav.tsx`	Sticky bottom navigation bar for mobile viewports (<768px).	Used in `MarketplaceLayout`.
`components/common/LocationModal.tsx`	Modal dialog allowing customers to select their active Goa delivery hub and Taluka.	Used in `MarketplaceLayout`.
`components/common/MarketplaceLayout.tsx`	Unified layout wrapper combining Header, Footer, MobileNav, LocationModal, and RoleSwitcherBar.	Wraps all public customer pages.
`components/common/ProtectedRoute.tsx`	Role-based route guard redirecting unauthenticated or unauthorized users to their proper view.	Wraps all portal routes in `App.tsx`.
`components/common/RoleSwitcherBar.tsx`	Demo utility toolbar allowing developers and testers to toggle between all 4 user roles instantly.	Embedded at the top of all pages.
`components/marketplace/ProductCard.tsx`	Dish card with Verified Origin badge, star rating pill, producer name, price, and inline Add/Qty controls.	Used on Home, Explore, and Vendor detail pages.
`components/marketplace/VendorCard.tsx`	Producer card with banner, avatar, SHG badge, authenticity score, and village origin.	Used on Home and Explore pages.
`components/marketplace/CategoryCard.tsx`	Category tile showing category icon, name, and dish count.	Used on Home page.
`components/marketplace/OrderTimeline.tsx`	Step-by-step visual tracker showing the 7-stage order lifecycle with timestamps and role badges.	Used in OrderDetail, VendorOrders, DeliveryDashboard, and AdminOrders.
`components/marketplace/ReviewModal.tsx`	Modal dialog for rating delivered dishes with 5-star selector, quick tag chips, and text feedback.	Used in OrderDetailPage.
`components/portals/PortalLayout.tsx`	Dashboard shell providing desktop sidebar, mobile responsive top bar, user summary, and portal badge.	Wraps all Vendor, Delivery Partner, and Admin pages.

4. Application Architecture & Execution Flow



Key Execution Milestones

1. **Mounting:** Vite serves `index.html`, which loads `src/main.tsx`. React initializes root mounting into `#root`.
2. **Storage Initialization:** When `AuthProvider` mounts, it invokes `initializeStorage()` from `storage.ts`. If `localStorage` is empty, initial seed datasets for products, vendors, demo users, orders, reviews, locations, and messages are written immediately.
3. **Context Hydration:** `LocationProvider` loads the default Goa hub (Panaji), `CartProvider` restores saved items, and `OrderProvider` fetches order history and establishes a cross-tab sync listener.
4. **Route Dispatch:** React Router matches the URL. Public marketplace views are wrapped in `MarketplaceLayout`. Portal views are protected by `ProtectedRoute`, which verifies permissions and renders `PortalLayout` with role-specific navigation.

5. Route and User-Role Mapping

Route Path	Page Component	Target Persona	Access Guard
`/`	`HomePage`	Customer / Public	Public
`/explore`	`ExplorePage`	Customer / Public	Public
`/product/:id`	`ProductDetailPage`	Customer / Public	Public
`/vendor/:id`	`VendorDetailPage`	Customer / Public	Public
`/cart`	`CartPage`	Customer / Public	Public
`/checkout`	`CheckoutPage`	Customer	Public (Guest inline auth)
`/orders`	`OrdersPage`	Customer	Requires Customer Login
`/orders/:id`	`OrderDetailPage`	Customer	Requires Customer Login
`/profile`	`CustomerProfilePage`	Customer	Requires Customer Login
`/vendor/register`	`VendorRegisterPage`	Vendor / Public	Public

Route Path	Page Component	Target Persona	Access Guard
`/vendor/dashboard`	`VendorDashboardPage`	Vendor / Admin	`vendor`, `admin`
`/vendor/products`	`VendorProductsPage`	Vendor / Admin	`vendor`, `admin`
`/vendor/orders`	`VendorOrdersPage`	Vendor / Admin	`vendor`, `admin`
`/vendor/earnings`	`VendorEarningsPage`	Vendor / Admin	`vendor`, `admin`
`/vendor/profile`	`VendorProfilePage`	Vendor / Admin	`vendor`, `admin`
`/delivery/tasks`	`DeliveryDashboardPage`	Delivery / Admin	`delivery`, `admin`
`/delivery/chats`	`DeliveryChatsPage`	Delivery / Admin	`delivery`, `admin`
`/delivery/profile`	`DeliveryProfilePage`	Delivery / Admin	`delivery`, `admin`
`/admin/dashboard`	`AdminDashboardPage`	Admin	`admin`
`/admin/vendors`	`AdminVendorsPage`	Admin	`admin`
`/admin/users`	`AdminUsersPage`	Admin	`admin`
`/admin/products`	`AdminProductsPage`	Admin	`admin`
`/admin/orders`	`AdminOrdersPage`	Admin	`admin`
`/admin/issues`	`AdminIssuesPage`	Admin	`admin`
`/admin/settlements`	`AdminSettlementsPage`	Admin	`admin`
`/admin/reports`	`AdminReportsPage`	Admin	`admin`

6. Feature-to-File Mapping

Feature Area	Page & UI Components	Service & Logic	Status
Marketplace Catalog	`HomePage.tsx`, `ProductCard.tsx`	`productService.ts`	FULLY FUNCTIONAL
Search & Filters	`ExplorePage.tsx`, `Header.tsx`	`productService.ts`	FULLY FUNCTIONAL
Origin Authenticity	`ProductDetailPage.tsx`	`productService.ts`	FULLY FUNCTIONAL
Single-Vendor Cart	`CartPage.tsx`, `CartContext.tsx`	`storage.ts`	FULLY FUNCTIONAL
Checkout & Confetti	`CheckoutPage.tsx`	`orderService.ts`	FULLY FUNCTIONAL
7-Stage Order Tracker	`OrderDetailPage.tsx`, `OrderTimeline.tsx`	`orderService.ts`	FULLY FUNCTIONAL
Verified Reviews	`ReviewModal.tsx`	`reviewService.ts`	FULLY FUNCTIONAL
Vendor Onboarding	`VendorRegisterPage.tsx`	`vendorService.ts`	FULLY FUNCTIONAL
Kitchen Orders Pipeline	`VendorOrdersPage.tsx`	`orderService.ts`	FULLY FUNCTIONAL
Vendor Product CRUD	`VendorProductsPage.tsx`	`productService.ts`	FULLY FUNCTIONAL
Vendor Earnings (5% Fee)	`VendorEarningsPage.tsx`	`orderService.ts`	FULLY FUNCTIONAL
Delivery Partner Tasks	`DeliveryDashboardPage.tsx`	`orderService.ts`	FULLY FUNCTIONAL
Delivery Order Chat	`DeliveryChatsPage.tsx`	`chatService.ts`	FULLY FUNCTIONAL
Admin Vendor Approvals	`AdminVendorsPage.tsx`	`vendorService.ts`	FULLY FUNCTIONAL
Admin Driver Dispatch	`AdminOrdersPage.tsx`	`orderService.ts`	FULLY FUNCTIONAL
Admin Disputes / Refunds	`AdminIssuesPage.tsx`	`adminService.ts`	FULLY FUNCTIONAL
Admin Settlements Payout	`AdminSettlementsPage.tsx`	`adminService.ts`	FULLY FUNCTIONAL
Analytics Reports	`AdminReportsPage.tsx`	Simulated UI metrics	PARTIAL (UI ONLY)

7. Data Storage & State Management

Rootify maintains 11 isolated keys in browser `localStorage`:

- `rootify_users`: User directory initialized from `DEMO_USERS`.

- `rootify_current_user`: Active user session record.
- `rootify_products`: Product catalog updated via vendor product management.
- `rootify_vendors`: Producer profiles updated via vendor onboarding and admin approvals.
- `rootify_orders`: Active and past orders with full 7-step timeline logs.
- `rootify_reviews`: Customer reviews triggering star rating recalculations.
- `rootify_messages`: Delivery partner messaging logs.
- `rootify_issues`: Dispute claims and refund audit records.
- `rootify_settlements`: 5% platform commission records generated upon order delivery.
- `rootify_selected_location`: Active customer Goa hub (default: Panaji).
- `rootify_cart`: Single-vendor shopping cart array.

8. UI, Styling, and Theme Guide

Color Palette Tokens (tailwind.config.js)

- **Brand Green (Primary):** `#155E52` (brand-600), `#114C42` (brand-700), `#0B2D28` (brand-900).
- **Warm Terracotta (Accent):** `#D95C3A` (terracotta-500), `#FAE8E2` (terracotta-100).
- **Warm Sand (Background):** `#FAF8F3` (sand-50 Canvas), `#EAE2D2` (sand-200 Border).
- **Charcoal (Text):** `#17332F` (charcoal-900).

Typography & Shadow Tokens

- **Heading Font:** `font-display` ('Outfit', sans-serif).
- **Body Font:** `font-sans` ('Plus Jakarta Sans', sans-serif).
- **Card Shadow:** `0 4px 14px rgba(23, 51, 47, 0.04)`.
- **Hover Shadow:** `0 10px 25px -3px rgba(21, 94, 82, 0.08)`.

9. Configuration and Dependencies

Dependency	Type	Role in Codebase
`react` / `react-dom`	Production (19.2.8)	UI rendering engine.
`react-router-dom`	Production (7.18.4)	SPA client-side routing.
`tailwindcss` / `postcss`	Production (3.4.17)	Utility CSS engine and Autoprefixer.
`lucide-react`	Production (1.51.0)	SVG icon component library.
`canvas-confetti`	Production (1.9.4)	Particle animation on order checkout.
`vite`	DevDependency (8.3.0)	Build tool and local dev server.
`typescript`	DevDependency (~6.0.2)	Static type checker.
`oxlint`	DevDependency (1.81.0)	Linter configuration.

10. Backend and Database Readiness Assessment

The Rootify codebase is exceptionally well-prepared for seamless backend integration:

- **Clean Interface Contracts:** Data logic is decoupled behind TypeScript interfaces (`IAuthService`, `IProductService`, `IOrderService`, `IVendorService`, etc.).
- **Zero UI Refactoring Required:** Replacing mock classes with HTTP `fetch()` or Axios clients will immediately connect all existing pages without altering UI component JSX.
- **Recommended Architecture:** Node.js/Express or Fastify backend connected to PostgreSQL (Prisma/Drizzle ORM) or Supabase, with S3/R2 storage for vendor verification documents.

11. Practical Editing Cheat Sheet

If I want to...	Which file should I inspect?	Why?
Change homepage hero banner or highlights	`src/pages/customer/HomePage.tsx`	Contains the hero banner JSX and featured product queries.
Change brand theme colors (green, terracotta, sand)	`tailwind.config.js`	Defines design token color variables.
Add a new Goan food category	`src/data/categories.ts` & `src/types/index.ts`	Defines category items, slugs, and icon mappings.
Add a new Goan village or delivery hub	`src/data/locations.ts`	Defines `GOA_LOCATIONS` and `TALUKAS` list.
Modify initial seed dishes or origin recipes	`src/data/products.ts`	Contains the 16 seed product objects and origin details.
Adjust delivery fee (₹40) or platform fee (₹15)	`src/context/CartContext.tsx` (lines 100–101)	Calculates subtotal and fee totals.
Adjust 5% platform commission rate	`src/services/orderService.ts` (line 234)	Calculates platform commission and vendor payable on delivery.
Add or change page routes	`src/App.tsx`	Registers all React Router `<Route>` entries and layouts.
Modify dish card layout or badge styling	`src/components/marketplace/ProductCard.tsx`	Renders the product card across all catalog views.
Change demo login accounts	`src/data/users.ts`	Seeds the 4 demo profiles (Customer, Vendor, Delivery, Admin).
Modify vendor verification document rules	`src/pages/vendor/VendorRegisterPage.tsx`	Defines document upload fields (Panchayat NOC, FSSAI, etc.).

12. Implementation Status, Codebase Observations & Next Steps

Confirmed Fully Working Features

- Complete Customer marketplace with full-text search, multi-facet filtering, and dish origin storytelling.
- Single-vendor shopping cart with vendor mismatch protection.
- Checkout with address selector, UPI/COD simulation, and confetti celebration.
- 7-stage live order tracking timeline.

• Verified purchase review system that dynamically recalculates average star ratings.
• Complete Vendor portal (incoming order pipeline, product CRUD, earnings ledger, profile).
• Complete Delivery Partner portal (task queue with status advancement, live order chat).
• Complete Admin desk (vendor verification/rejection with feedback, catalog moderation, driver dispatch, dispute resolution, settlements).
• One-click demo role switcher bar.

Codebase Observations & Refactoring Recommendations

• **Duplicate Class Definition:** `src/services/chatService.ts` (lines 52–110) contains an extra definition of `IAdminService` and `adminService`, which is also defined in `src/services/adminService.ts`. *Recommendation:* In future cleanup, remove lines 52–110 from `chatService.ts`.
• **Unused File:** `src/App.css` is a legacy Vite template remnant. Can be safely deleted.
• **Hardcoded Analytics:** `AdminReportsPage.tsx` currently displays static category sales percentages. *Recommendation:* Connect to dynamic computation from `useOrders()`.

13. Summary Conclusion

Rootify is a production-grade, aesthetically tailored, regional marketplace web application. The codebase cleanly isolates data access within a modular service layer, allowing seamless future backend integration without any disruption to the existing UI or user flows.